



Proyecto Final

Carlos Jarero

Objetivos

El objetivo principal de este estudio es construir un modelo de predicción de mortalidad y complicaciones mortales basado en datos clínicos y demográficos disponibles en el momento del ingreso

- Vivo
- Muertos
 - Shock cardiogénico
 - Edema pulmonar
 - Ruptura miocárdica
 - Progresión de insuficiencia cardíaca congestiva
 - Tromboembolismo
 - Asistolia
 - Fibrilación ventricular

Fuente de datos



Myocardial infarction complications Database

[Cite](#)[Download all \(2.14 MB\)](#)[Share](#)[Embed](#)[+ Collect](#)

Version 3 ▾ Dataset posted on 2020-07-07, 07:54 authored by S.E. Golovenkin, [Alexander Gorban](#), [Evgeny Mirkes](#),

V.A. Shulman, D.A. Rossiev, P.A. Shesternya, S.Yu. Nikulina, Yu.V. Orlova, M.G. Dorrer

This dataset contains a database for Myocardial Infarction complication prediction (csv file) , collected in 1992-1995 in Krasnoyarsk with associated description (pdf file). Descriptive statistics of all database attributes are presented in Descriptive Statistics file.

USAGE METRICS [↗](#)

20,263
views

5,491
downloads

12
[citations](#) [↗](#)



HISTORY

- **2020-03-30** - First online date
- **2020-07-07** - Posted date

CATEGORIES

	ID	AGE	SEX	INF_ANAM	STENOK_AN	FK_STENOK	IBS_POST	IBS_NASL	GB	SIM_GIPERT	DLIT_AG	ZSN_A	nr_11
0	1	77.0	1	2.0	1.0	1.0	2.0	NaN	3.0	0.0	7.0	0.0	0.0
1	2	55.0	1	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	3	52.0	1	0.0	0.0	0.0	2.0	NaN	2.0	0.0	2.0	0.0	0.0
3	4	68.0	0	0.0	0.0	0.0	2.0	NaN	2.0	0.0	3.0	1.0	0.0
4	5	60.0	1	0.0	0.0	0.0	2.0	NaN	3.0	0.0	7.0	0.0	0.0
5	6	64.0	1	0.0	1.0	2.0	1.0	NaN	0.0	0.0	0.0	0.0	0.0
6	7	70.0	1	1.0	1.0	2.0	1.0	NaN	2.0	0.0	7.0	1.0	0.0
7	8	65.0	1	0.0	1.0	1.0	2.0	NaN	2.0	0.0	7.0	0.0	0.0
8	9	60.0	1	0.0	0.0	0.0	2.0	NaN	2.0	0.0	6.0	0.0	0.0
9	10	77.0	0	2.0	0.0	0.0	0.0	NaN	3.0	0.0	6.0	1.0	0.0
10	11	71.0	1	0.0	0.0	0.0	0.0	NaN	0.0	0.0	0.0	0.0	0.0
11	12	50.0	0	0.0	0.0	0.0	0.0	NaN	2.0	0.0	3.0	0.0	0.0
12	13	60.0	1	1.0	0.0	0.0	2.0	NaN	2.0	0.0	2.0	0.0	0.0
13	14	57.0	1	0.0	0.0	0.0	2.0	0.0	2.0	0.0	6.0	0.0	0.0
14	15	55.0	1	1.0	1.0	2.0	2.0	NaN	2.0	0.0	1.0	0.0	0.0
15	16	57.0	1	3.0	0.0	0.0	0.0	NaN	2.0	0.0	6.0	0.0	0.0
16	17	52.0	1	0.0	6.0	2.0	1.0	NaN	2.0	0.0	7.0	0.0	0.0
17	18	52.0	0	1.0	1.0	2.0	2.0	NaN	2.0	0.0	7.0	1.0	0.0
18	19	54.0	1	0.0	1.0	3.0	2.0	NaN	2.0	0.0	2.0	0.0	0.0
19	20	71.0	0	0.0	1.0	4.0	2.0	NaN	0.0	0.0	0.0	0.0	0.0
20	21	70.0	0	0.0	0.0	0.0	2.0	NaN	0.0	0.0	0.0	0.0	0.0

Eliminación de columnas innecesarias

```
"IBS_NASL" # Coronaria patología Hereditaria
, "S_AD_ORIT" # TAS ICU
, "D_AD_ORIT" # TAS ICU
, "O_L_POST" # UCI
, "K_SH_POST" # UCI
, "MP_TP_POST" # UCI o prehospital pero no se pueden distinguir
, "SVT_POST" # UCI o prehospital pero no se pueden distinguir
, "GT_POST" # UCI o prehospital pero no se pueden distinguir
, "FIB_6_POST" # UCI o prehospital pero no se pueden distinguir
, "GIPO_K" # Potasio bajo
, "GIPER_NA" # Invremento de solido
, "ALT_BLOOD" # ALT no es importante
, "AST_BLOOD" # AST no es importante
, "L_BLOOD" # Leucos no son importanes
, "ROE" # VSG no son importanes
, "NITR_S" # Parámetros de UCI fuera del alcance
, "NA_R_1_n" # Parámetros de UCI fuera del alcance
, "NA_R_2_n" # Parámetros de UCI fuera del alcance
, "NA_R_3_n" # Parámetros de UCI fuera del alcance
, "NOT_NA_1_n" # Parámetros de UCI fuera del alcance
, "NOT_NA_2_n" # Parámetros de UCI fuera del alcance
, "NOT_NA_3_n" # Parámetros de UCI fuera del alcance
, "LID_S_n" # Parámetros de UCI fuera del alcance
, "B_BLOK_S_n" # Parámetros de UCI fuera del alcance
, "ANT_CA_S_n" # Parámetros de UCI fuera del alcance
, "GEPAR_S_n" # Parámetros de UCI fuera del alcance
```

```
, "ASP_S_n" # Parámetros de UCI fuera del alcance
, "TIKL_S_n" # Parámetros de UCI fuera del alcance
, "TRENT_S_n" # Parámetros de UCI fuera del alcance
, "FIBR_PREDS" # Complicaciones no mortales, fuera del alcance
, "PREDS_TAH" # Complicaciones no mortales, fuera del alcance
, "JELUD_TAH" # Complicaciones no mortales, fuera del alcance
, "FIBR_JELUD" # Complicaciones no mortales, fuera del alcance
, "A_V_BLOK" # Complicaciones no mortales, fuera del alcance
, "OTEK_LANC" # Complicaciones no mortales, fuera del alcance
, "RAZRIV" # Complicaciones no mortales, fuera del alcance
, "DRESSLER" # Complicaciones no mortales, fuera del alcance
, "ZSN" # Complicaciones no mortales, fuera del alcance
, "REC_IM" # Complicaciones no mortales, fuera del alcance
, "P_IM_STEN" # Complicaciones no mortales, fuera del alcance
, "R_AB_1_n" # Parámetros de dolor, fuera del alcance
, "R_AB_2_n" # Parámetros de dolor, fuera del alcance
, "R_AB_3_n" # Parámetros de dolor, fuera del alcance
, "NA_KB" # Parámetro sobre tratamiento en urgencias, está fuera del alcance
, "LID_KB" # Parámetro sobre tratamiento en urgencias, está fuera del alcance
, "endocr_03" # Parámetros clínicamente poco importantes para un infarto
, "zab_leg_01" # Parámetros clínicamente poco importantes para un infarto
, "zab_leg_02" # Parámetros clínicamente poco importantes para un infarto
, "zab_leg_03" # Parámetros clínicamente poco importantes para un infarto
, "zab_leg_04" # Parámetros clínicamente poco importantes para un infarto
, "zab_leg_06" # Parámetros clínicamente poco importantes para un infarto
```

Tratamiento de valores nulos

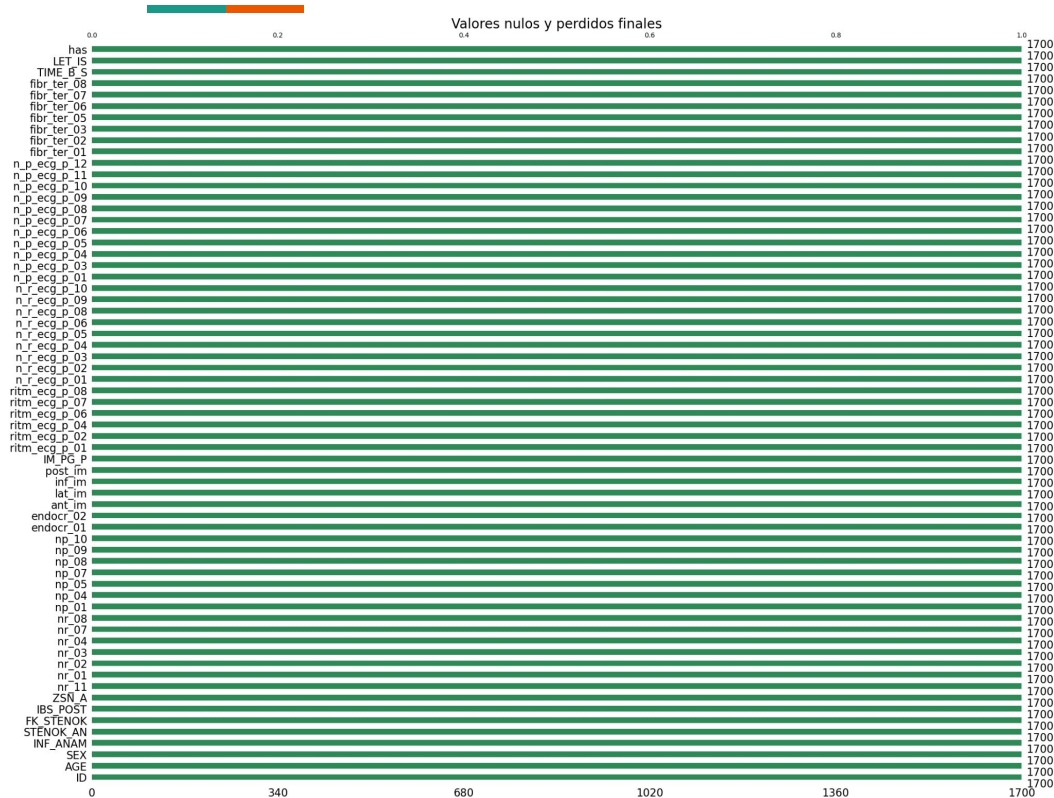


```
COLUMNAS_VALORES_PERDIDOS = [
    "K_BLOOD"
    , "NA_BLOOD"
    , "KFK_BLOOD"
    , "NOT_NA_KB"
    , "S_AD_KBRIG"
    , "D_AD_KBRIG"
]
```

GB
SIM_GIPERT
DLIT_AG

Moda

Tratamiento de valores nulos



Union de columnas

```
FIBRINOLITICOS = [
    "fibr_ter_01"
    , "fibr_ter_02"
    , "fibr_ter_03"
    , "fibr_ter_05"
    , "fibr_ter_06"
    , "fibr_ter_07"
    , "fibr_ter_08"
]

df["tratamiento_fibrinolitico"] = df[FIBRINOLITICOS].any(axis=1).astype(int)

# Removemos las columnas
df = df.drop(columns=FIBRINOLITICOS)

# Creamos una columna de uso de tratamiento antecedente_arritmia
ANTECEDENTE_ARRITMIA = [
    "nr_11"
    , "nr_01"
    , "nr_02"
    , "nr_03"
    , "nr_04"
    , "nr_07"
    , "nr_08"
]

df["antecedente_arritmia"] = df[ANTECEDENTE_ARRITMIA].any(axis=1).astype(int)

# Removemos las columnas
df = df.drop(columns=ANTECEDENTE_ARRITMIA)

# Creamos una columna de uso de tratamiento antecedente_bloqueo
ANTECEDENTE_BLOQUEO = [
    "np_01"
    , "np_04"
    , "np_05"
    , "np_07"
    , "np_08"
    , "np_09"
    , "np_10"
]

df["antecedente_bloqueo"] = df[ANTECEDENTE_BLOQUEO].any(axis=1).astype(int)

# Removemos las columnas
df = df.drop(columns=ANTECEDENTE_BLOQUEO)
```

```
ecg_rimto_grupo = df[df[["ritm_ecg_p_01",
    "ritm_ecg_p_07",
    "ritm_ecg_p_08",
    "ritm_ecg_p_02",
    "ritm_ecg_p_04",
    "ritm_ecg_p_06"
]].eq(1).sum(axis=1) > 1]

# Validamos si las categorias n_r_ecg_p_## son mutuamente excluyentes
ecg_hallazgos_grupo = df[df[["n_r_ecg_p_01"
    , "n_r_ecg_p_02"
    , "n_r_ecg_p_03"
    , "n_r_ecg_p_04"
    , "n_r_ecg_p_05"
    , "n_r_ecg_p_06"
    , "n_r_ecg_p_08"
    , "n_r_ecg_p_09"
    , "n_r_ecg_p_10"
]].eq(1).sum(axis=1) > 1]

# Validamos si las categorias n_p_ecg_p_## son mutuamente excluyentes
ecg_bloqueos_grupo = df[df[["n_p_ecg_p_01"
    , "n_p_ecg_p_03"
    , "n_p_ecg_p_04"
    , "n_p_ecg_p_05"
    , "n_p_ecg_p_06"
    , "n_p_ecg_p_07"
    , "n_p_ecg_p_08"
    , "n_p_ecg_p_09"
    , "n_p_ecg_p_10"
    , "n_p_ecg_p_11"
    , "n_p_ecg_p_12"
]].eq(1).sum(axis=1) > 1]
```


Generacion de categorías

```
COLUMNAS_BINARIAS = [
    "SEX"
    , "endocr_01"
    , "endocr_02"
    , "ant_im"
    , "lat_im"
    , "inf_im"
    , "post_im"
    , "IM_PG_P"
    , "has"
    , "tratamiento_fibrinolitico"
    , "antecedente_arritmia"
    , "antecedente_bloqueo"
]

for col in COLUMNAS_BINARIAS:
    df[col] = df[col].map(categoria_binaria)
    df[col] = pd.Categorical(df[col])

df["muerte"] = df["muerte"].map({0: "Vivo", 1: "Muerto"})
df["muerte"] = pd.Categorical(df["muerte"])

df["INF_ANAM"] = df["INF_ANAM"].map({0: "Cero", 1:"Uno",2:"Dos", 3:"Tres o más" })
df["INF_ANAM"] = pd.Categorical(df["INF_ANAM"], ordered=True)

df["STENOK_AN"] = df["STENOK_AN"].map({0: "Nunca"
    , 1:"Durante el último año"
    , 2:"Hace un año"
    , 3:"Hace 2 años"
    , 4:"Hace 3 años"
    , 5:"Hace 4-5 años"
    , 6:"Más de 5 años"
})
df["STENOK_AN"] = pd.Categorical(df["STENOK_AN"], ordered=True)
```

```
df["IBS_POST"] = df["IBS_POST"].map({0: "No hubo enfermedad coronaria"
    , 1: "Angina de pecho de esfuerzo"
    , 2: "Angina de pecho inestable"
})
df["IBS_POST"] = pd.Categorical(df["IBS_POST"], ordered=True)

df["ZSN_A"] = df["ZSN_A"].map({
    0: "No hay insuficiencia cardíaca crónica",
    1: "Etapa I",
    2: "Etapa IIA (insuficiencia cardíaca por disfunción sistólica del ventrículo derecho)",
    3: "Etapa IIA (insuficiencia cardíaca por disfunción sistólica del ventrículo izquierdo)",
    4: "Etapa IIB (insuficiencia cardíaca por disfunción sistólica de ambos ventrículos)"
})
df["ZSN_A"] = pd.Categorical(df["ZSN_A"]) # No puede ser ordinal por que combinan IIA derecha e izq

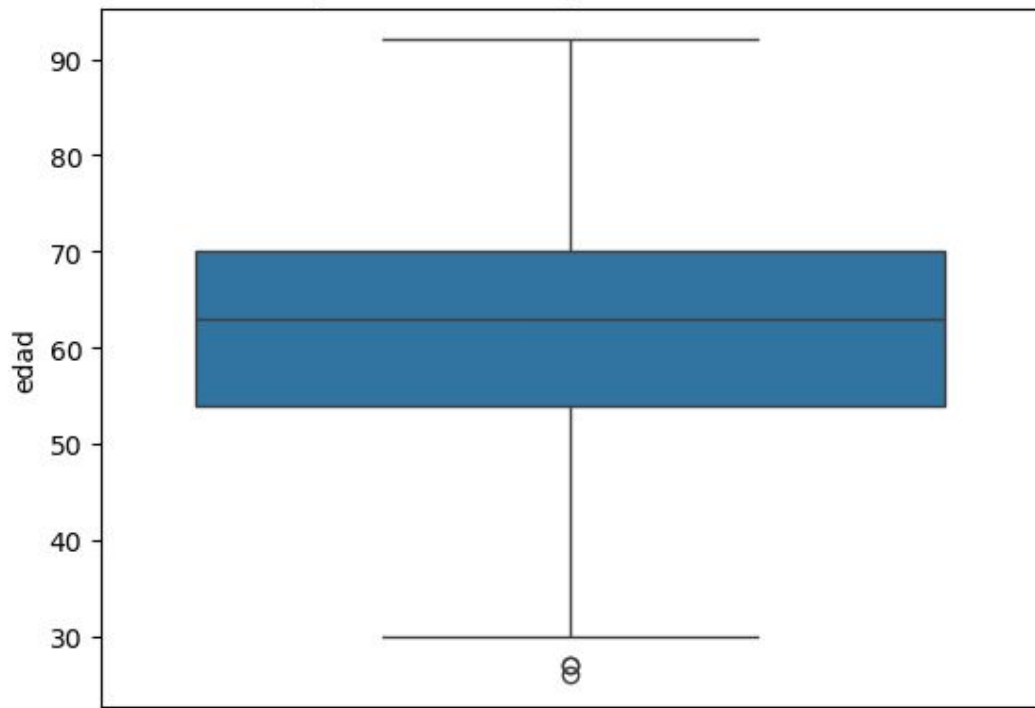
df["TIME_B_S"] = df["TIME_B_S"].map({
    1: "Menos de 2 horas",
    2: "2-4 horas",
    3: "4-6 horas",
    4: "6-8 horas",
    5: "8-12 horas",
    6: "12-24 horas",
    7: "Más de 1 día",
    8: "Más de 2 días",
    9: "Más de 3 días"
})
df["TIME_B_S"] = pd.Categorical(df["TIME_B_S"], ordered=True)

df["LET_IS"] = df["LET_IS"].map({
    0: "Vivo",
    1: "Shock cardiogénico",
    2: "Edema pulmonar",
    3: "Ruptura miocárdica",
    4: "Progresión de insuficiencia cardíaca",
    5: "Tromboembolismo",
    6: "Asistolia",
    7: "Fibrilación ventricular"
})
df["LET_IS"] = pd.Categorical(df["LET_IS"])
```

Outliers RIQ



Boxplot de la edad para buscar outliers

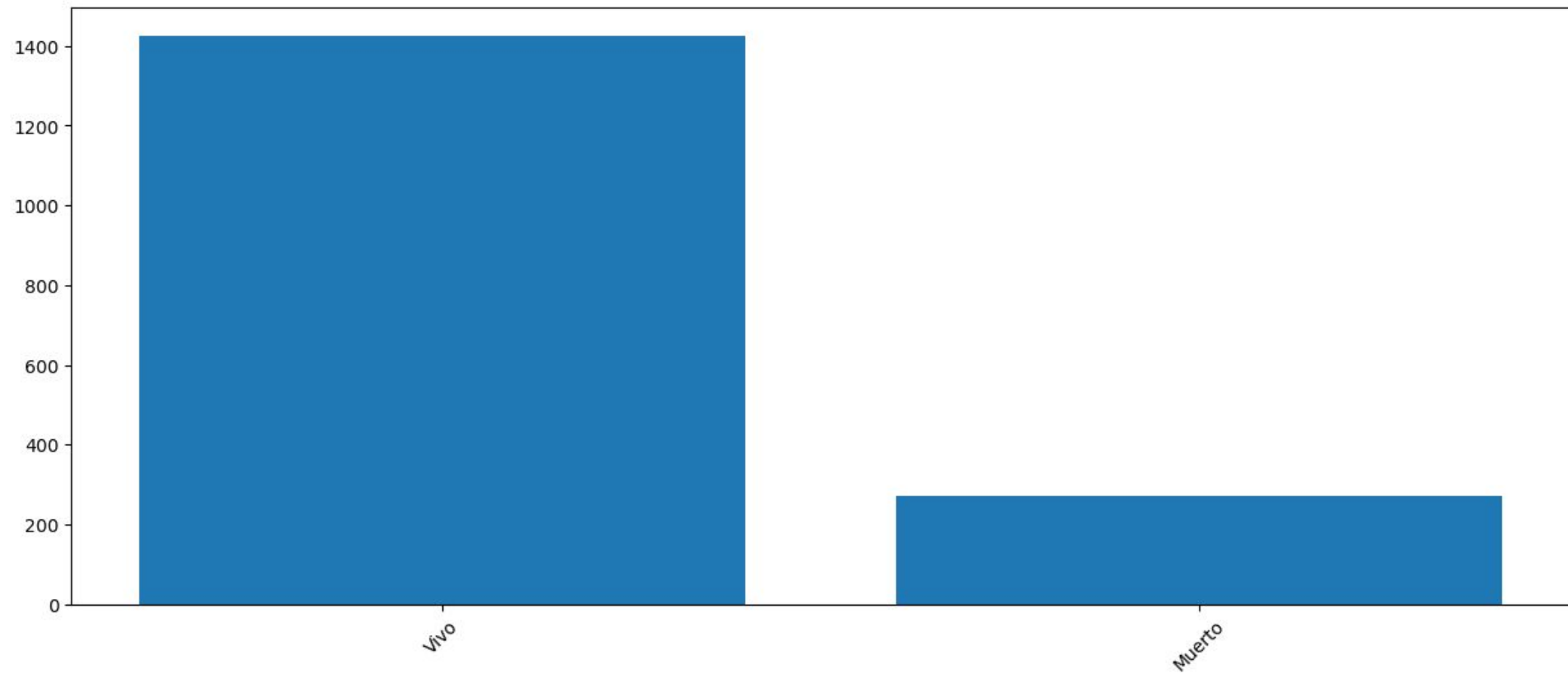


id		edad
312	313	27.0
329	330	27.0
987	988	26.0

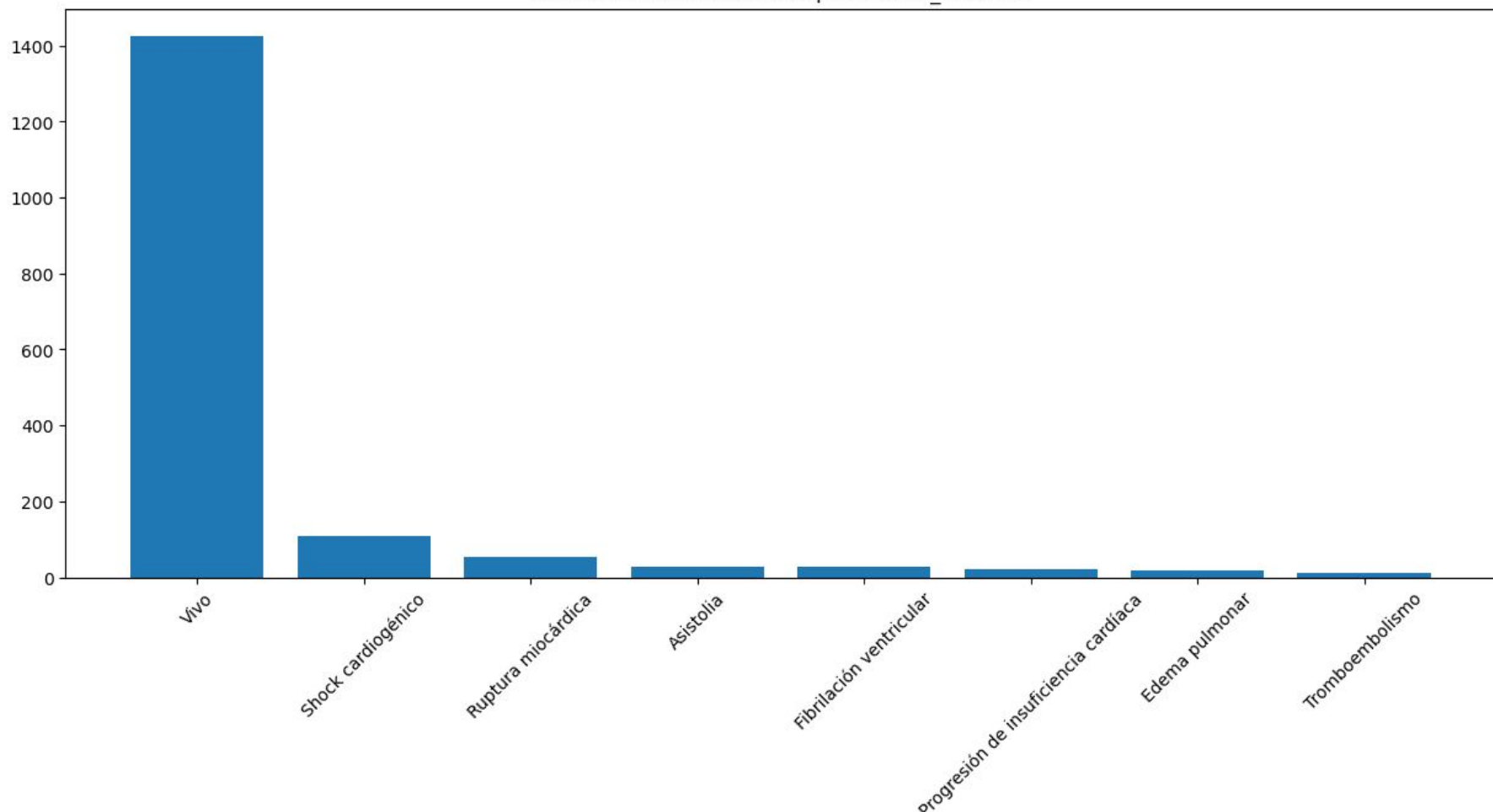
Nuevas de columnas

```
<class 'pandas.core.frame.DataFrame'>
Index: 1697 entries, 0 to 1699
Data columns (total 25 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   id                                         1697 non-null   int64
1   edad                                      1697 non-null   float64
2   sexo                                      1697 non-null   category
3   has                                        1697 non-null   category
4   cantidad_im_previos                      1697 non-null   category
5   antecedente_angina                      1697 non-null   category
6   cf_ultimo_ano                            1697 non-null   category
7   enfermedad_coronaria_reciente          1697 non-null   category
8   icc                                       1697 non-null   category
9   dm                                        1697 non-null   category
10  obesidad                                 1697 non-null   category
11  antecedente_arritmia                     1697 non-null   category
12  antecedente_bloqueo                      1697 non-null   category
13  im_anterior                             1697 non-null   category
14  im_lateral                              1697 non-null   category
15  im_inferior                             1697 non-null   category
16  im_posterior                            1697 non-null   category
17  im_vd                                   1697 non-null   category
18  tiempo_evolucion                        1697 non-null   category
19  tratamiento_fibrinolitico               1697 non-null   category
20  ritmo_ecg                              1697 non-null   category
21  hallazgos_ecg_ingreso                   1697 non-null   category
22  bloque_ecg_ingreso                     1697 non-null   category
23  muerte                                  1697 non-null   category
24  complicaciones_mortales                  1697 non-null   category
dtypes: category(23), float64(1), int64(1)
memory usage: 82.4 KB
```

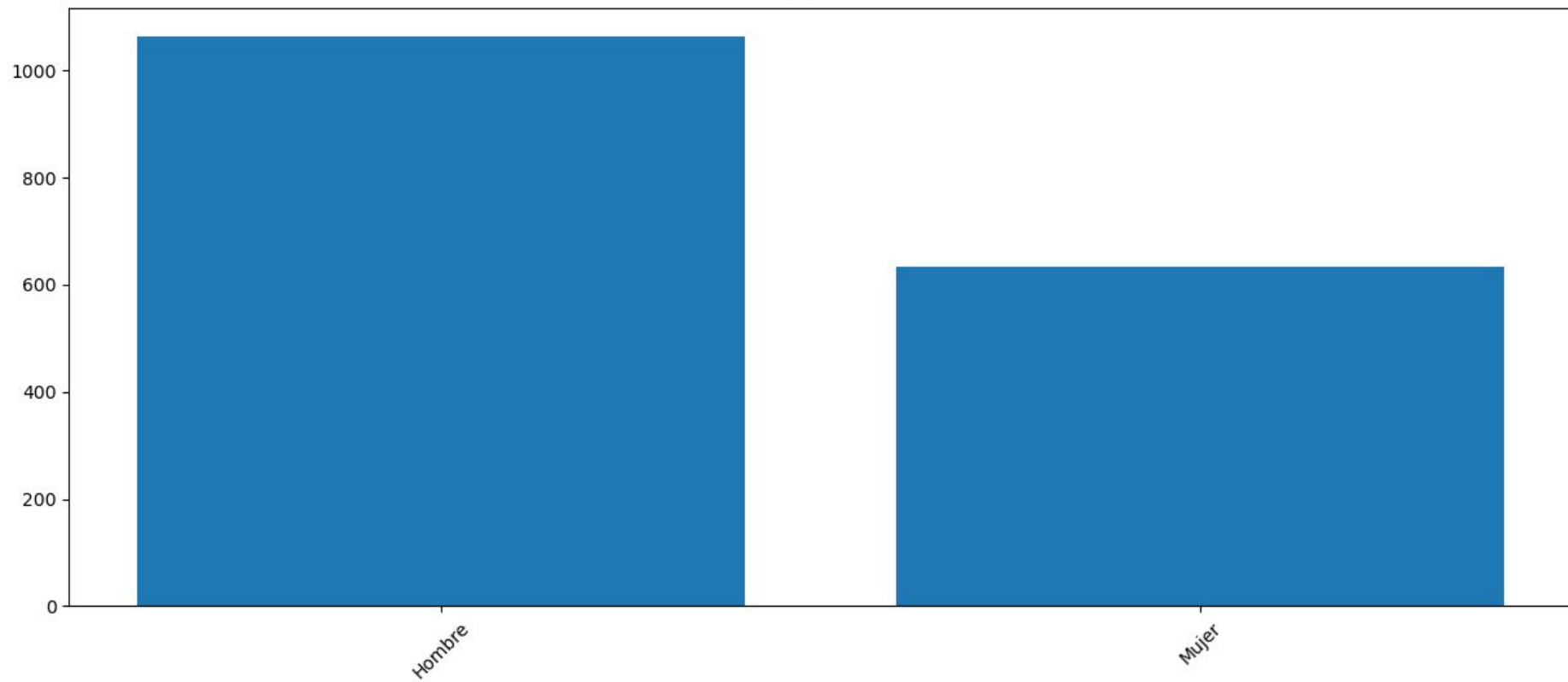
Frecuencias de variable muerte



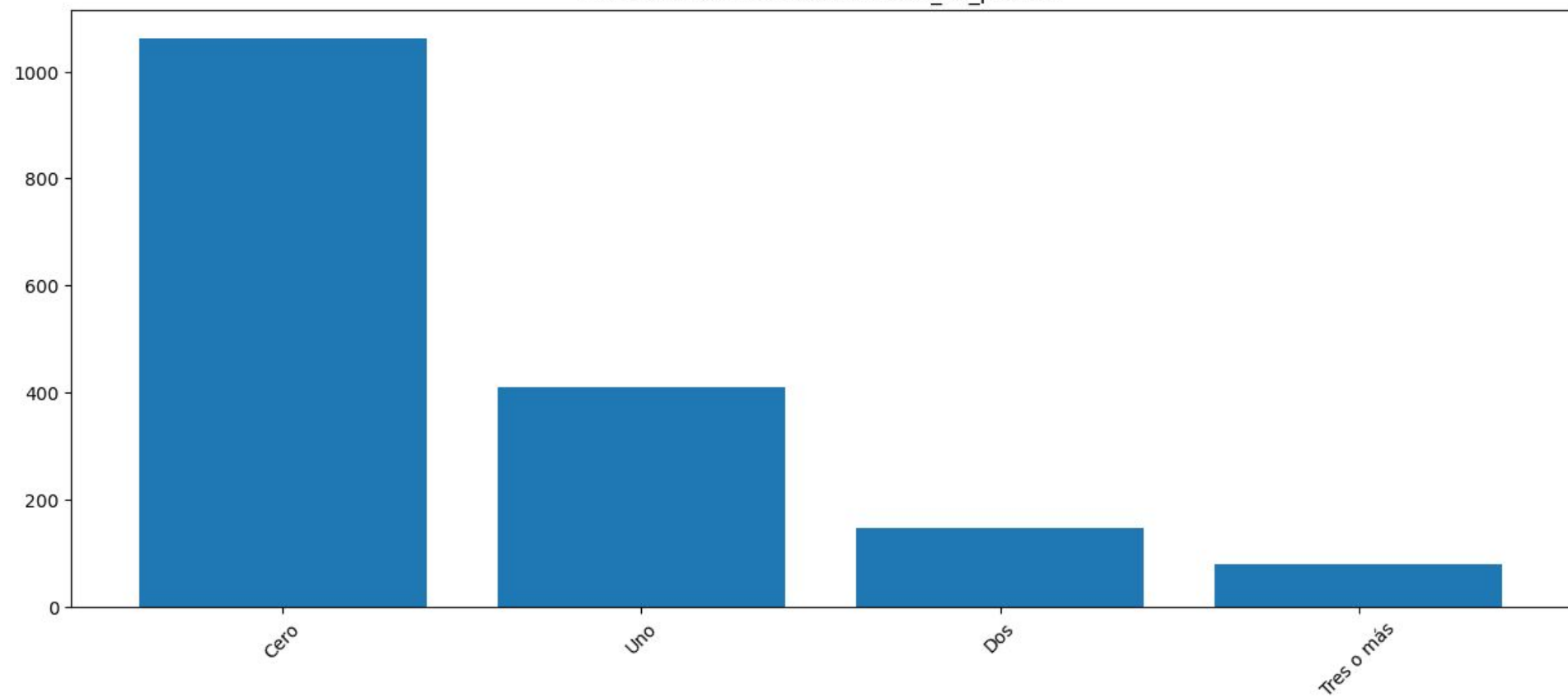
Frecuencias de valriable complicaciones_mortales



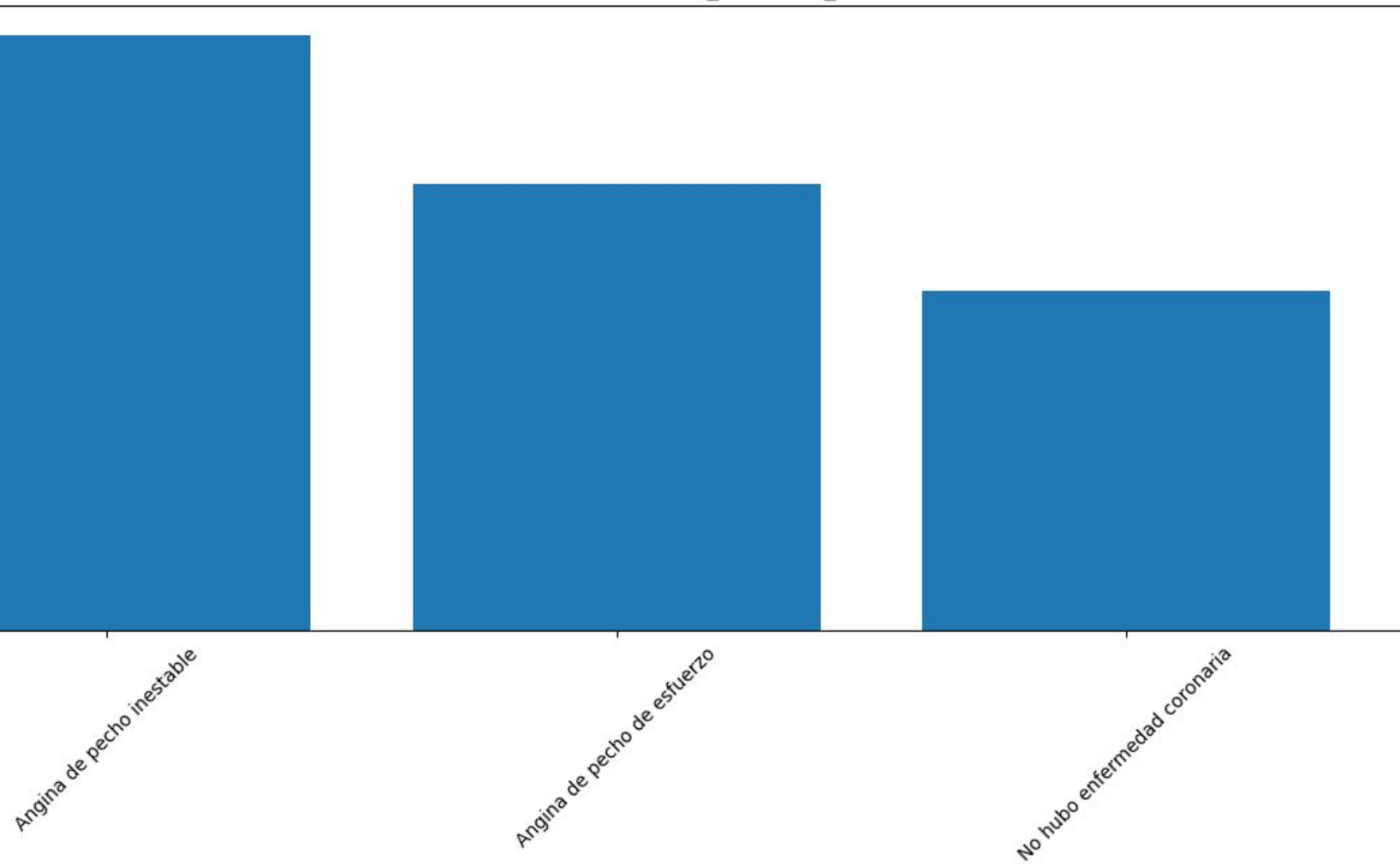
Frecuecias de valriable sexo



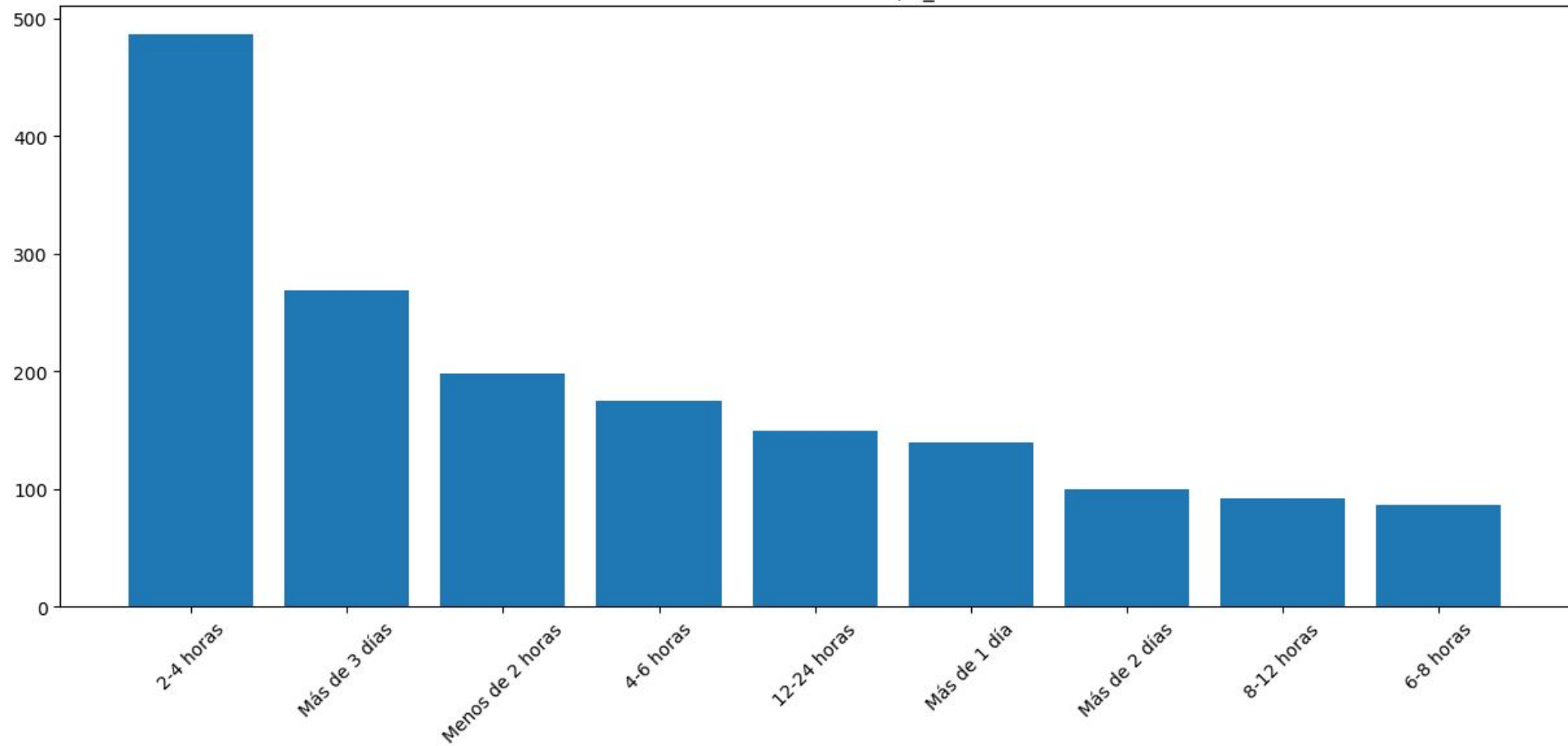
Frecuecias de valriable cantidad_im_previos



Frecuencias de valriable enfermedad_coronaria_reciente

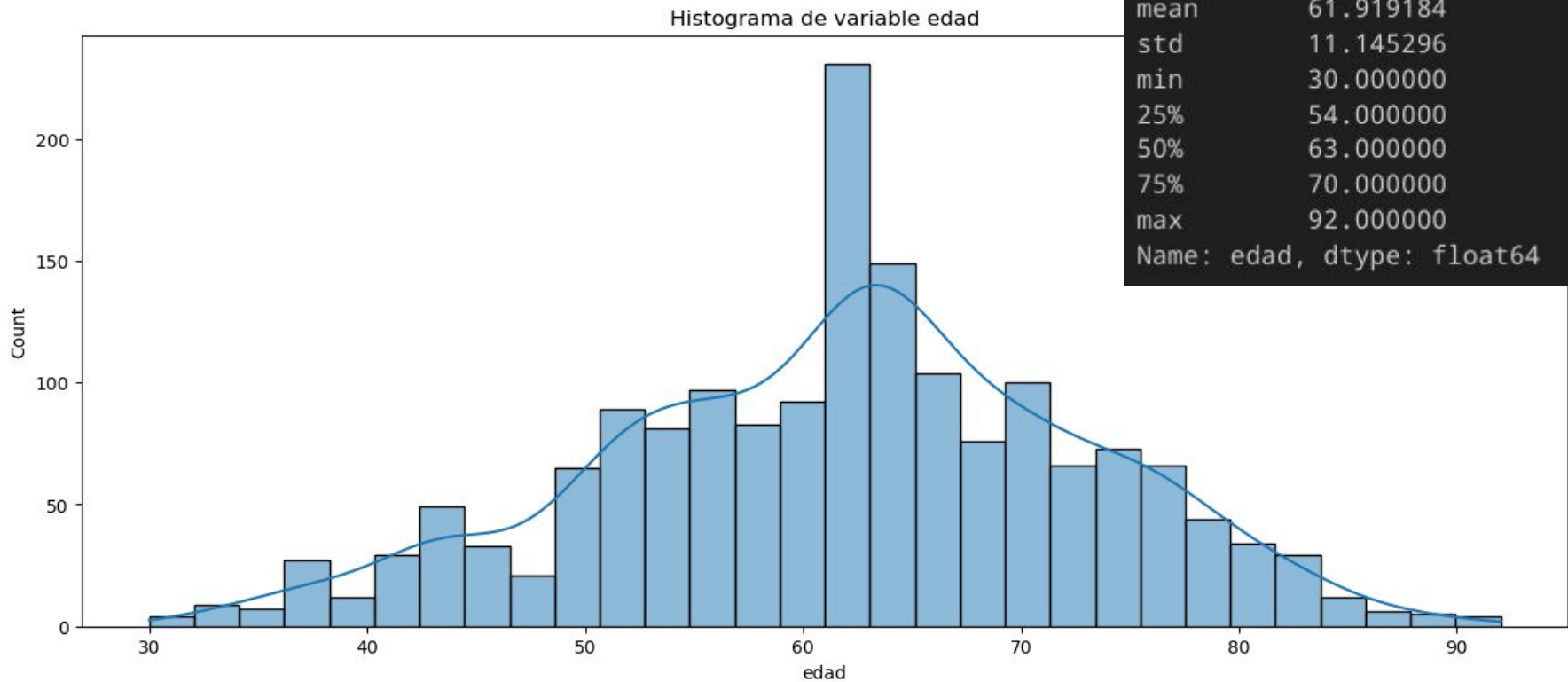


Frecuecias de valriable tiempo_evolution



Edad

sw = 7.703884682129431e-07



Edad (BoxCox)

sw = 4.742690021304255e-05



Histograma de variable edad boxcox [1.3283017342900956]

